ReBAC V2.1 — Production Rule Engine

> ***Branch:*** `ReBAC-V2.1` | ***Builds on:*** *V2 Recursive Authorization*

The Problems V2 Left Unsolved

Problem	Risk
No cycle detection	Server crash on cyclic resource graphs
No memoization	Exponential re-traversal on shared ancestors
No evaluation trace	Impossible to debug why access was granted/denied
No operator algebra	Only implicit OR — no AND, NOT

V2.1 is a **hardening sprint** — it does not add features but makes the engine production-safe and observable.

V2.1 Architecture

RuleGroup — The IR for Permission Expressions

```
interface RuleGroup {
  operator: "OR" | "AND" | "NOT";
  rules: (Rule | RuleGroup)[];   // composable tree
}

interface Rule {
  relation: string;         // direct: "doctor_of"
```

```
  permission?: string;      // recursive: "contains" → "view"
}
```

This tree structure is the **Intermediate Representation (IR)** between the schema definition and runtime evaluation. It maps directly to a boolean expression tree.

Dual Safety Mechanisms

```
interface EvaluationContext {
  processing: Set<string>;    // cycle detection — O(1) lookup
  memo: Map<string, boolean>; // memoization — dynamic programming
  trace: TraceStep[];         // observability
}
```

Cycle detection uses a `processing` set (3-color DFS equivalent):

- Before visiting a node → add to `processing`
- If node already in `processing` → cycle detected → return false
- After visiting → remove from `processing`

Memoization uses a `memo` map keyed on `userId:resourceId:permission`:

```
First call: userId=1, resourceId=5, permission="view" → traverse DFS
→ cache result
Second call: same key → return cached result instantly O(1)
```

Decomposed RuleEngine Methods

```
RuleEngine
 ├── checkPermission()      ← top-level with memo + cycle checks
 ├── evaluateOperator()     ← handles OR/AND/NOT operator nodes
 ├── evaluateRule()         ← dispatches to direct or recursive
 ├── evaluateDirect()       ← single SQL lookup
 └── evaluateRecursive()    ← DFS upward through resource hierarchy
```

Short-circuit evaluation is critical for OR:

```
OR [doctor_of, assigned_nurse, contains.view]
→ evaluate doctor_of → true → STOP (don't evaluate remaining
```

```
branches)
```

Critical Review

✅ What V2.1 Gets Right

1. **Memoization is the correct DP optimization.** The memo key `userId:resourceId:permission` is complete and unambiguous.
2. **Cycle detection via processing set** correctly implements the 3-color DFS safety check (white/gray/black equivalent).
3. **RuleGroup IR** correctly models boolean algebra over authorization rules — this is equivalent to Zanzibar's *permission formula* AST.
4. **Evaluation trace** is the precursor to Zanzibar's *check trace* debugging output.

⚠️ What V2.1 Is Still Missing

Missing: Request-scoped context The `memo` map must be **request-scoped**, not singleton. If a singleton memo is used, stale cached results from one request will bleed into another. V2.1 correctly instantiates a new context per `checkPermission` call.

Missing: Memo invalidation on tuple writes When a relationship is written (`Raj is now doctor_of Patient202`), any cached results referencing Patient202 must be invalidated. V2.1 has no cache invalidation strategy.

Missing: Concurrency safety Node.js is single-threaded but async. If two concurrent `checkPermission` calls share state (e.g., a singleton memo), race conditions emerge.

Missing: Timeout / deadline propagation Long-running recursive traversals have no timeout. A malicious or malformed resource graph could cause DoS.

❌ Incorrect Assumption: Memoization Correctness

The memo assumes **permission results are pure functions of (userId, resourceId, permission)**. This is only true if the database state doesn't change between memo writes and reads. In a system with concurrent tuple writes, this assumption breaks.

SpiceDB's solution: ZedTokens — each response includes a consistency token. Lookups that must see a write use the token to bypass stale cache results.

Comparison with Industry

Feature	V2.1	SpiceDB	OpenFGA
Memoization	✅ in-memory	✅ distributed	✅
Cycle detection	✅	✅	✅
Cache invalidation	❌	✅ ZedTokens	✅
Timeout	❌	✅	✅
Parallel evaluation	❌	✅ goroutines	✅
Evaluation trace	✅ basic	✅ full	✅

Summary

V2.1 transforms the V2 prototype into a production-capable engine by adding memoization (dynamic programming), cycle detection (3-color DFS), and a composable RuleGroup IR for boolean operator evaluation. The engine is now **safe** but not yet **dynamic** — the permission rules are still hardcoded in TypeScript. V2.2 solves this with a custom compiler.